Machine Learning Notes

Page 1 — Linear Regression Basics

1) Definition

Linear Regression is a supervised learning algorithm used to predict a **continuous** (target) value based on one or more independent (features) variables.

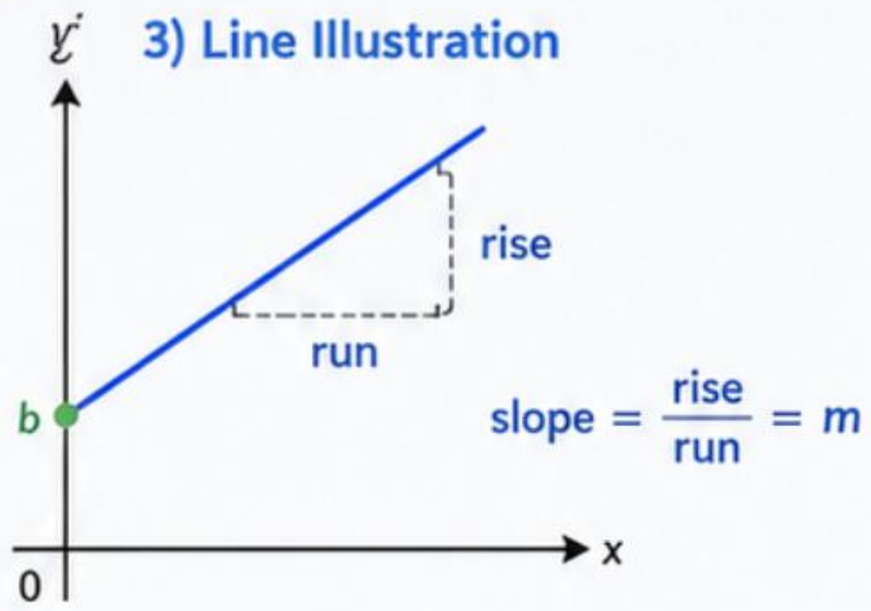
2) Linear Equation

$$y = mx + b$$

m = slope (rate of change)

b = intercept (value of *y* when *x* = 0)

3) Line Illustration



4) scikit-learn (sklearn) Overview

5) Import

```
from sklearn.linear_model
import LinearRegression
```

6) Linear Regression Workflow

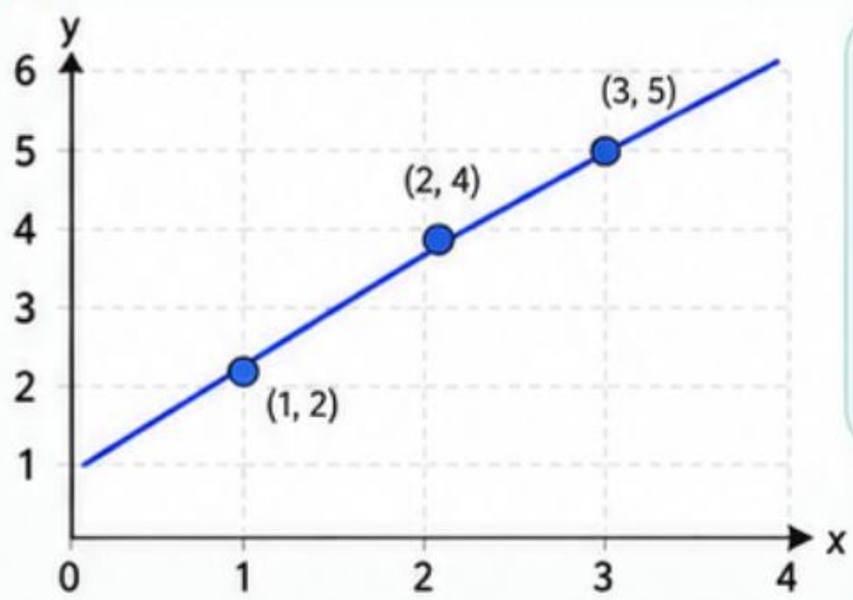
1. Import all required libraries
2. Read data into a DataFrame
3. Preprocess data: handle null values, noisy data, and data-type mismatches
4. Scale features: StandardScaler, MinMaxScaler, RobustScaler, Normalization
5. Define X (features) and y (target), then split into training and testing sets
6. Create the model object (e.g., LinearRegression())
7. Fit the model on X_train and y_train
8. Evaluate the model (e.g., R^2 score, MSE, MAE, RMSE)
9. Predict for unknown/new data

7) Example Data

Index	x (feature)	y (target)
1	1	2
2	2	4
3	3	5

We will use this data to fit a line of the form $y = mx + b$.

8) Scatter Plot with Best-Fit Line



The line summarizes the relationship between x and y.

➡ Detailed calculation of slope (*m*) and intercept (*b*) continues on Page 2.

Machine Learning Notes

Page 2 — Linear Regression Calculations

1 Dataset

x (feature)	y (target)
1	2
2	4
3	5

We will fit the line $y = mx + b$.

2 Matrix Form

We add a bias term (column of 1's).

$$X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{bmatrix} \qquad \theta = \begin{bmatrix} b \\ m \end{bmatrix}$$

$$y = \begin{bmatrix} 2 \\ 4 \\ 5 \end{bmatrix}$$

$$X\theta = y$$

3 Ordinary Least Squares (OLS)

$$\theta = (X^T X)^{-1} X^T y$$

Equivalently (normal equation):

$$(X^T X)\theta = X^T y$$

This gives the values of b (intercept) and m (slope) that minimize the sum of squared errors.

4 Compute $X^T X$ and $X^T y$

$$X^T X = \begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix} \qquad X^T y = \begin{bmatrix} 11 \\ 25 \end{bmatrix}$$

So the normal equations are:

$$\begin{bmatrix} 3 & 6 \\ 6 & 14 \end{bmatrix} \begin{bmatrix} b \\ m \end{bmatrix} = \begin{bmatrix} 11 \\ 25 \end{bmatrix} \rightarrow \begin{cases} 3b + 6m = 11 \\ 6b + 14m = 25 \end{cases}$$

5 Solve for b and m

$$m = 1.5$$

$$b \approx 0.667 \ \left(\approx \frac{2}{3}\right)$$

Final fitted model:

$$y = 0.667 + 1.5x$$

6 Prediction Example

For $x = 4$:

$$\hat{y} = 0.667 + 1.5(4) = 6.667$$

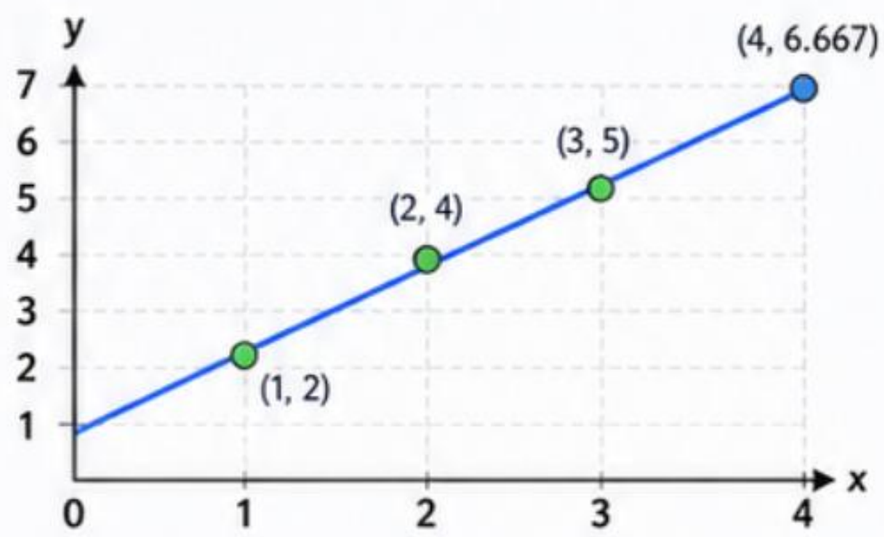
➡ Predicted $y \approx 6.667$

7 Fitted Values, Errors and Squared Errors

x	Actual y	Predicted $\hat{y}$ $(= 0.667 + 1.5x)$	Error $(y - \hat{y})$	Squared Error $(y - \hat{y})^2$
1	2	2.167	−0.167	0.0278
2	4	3.667	0.333	0.1111
3	5	5.167	−0.167	0.0278

Mean Squared Error (MSE) MSE = 0.0556 ($\approx$ 0.055)

8 Best-Fit Line (Illustration)



Fitted line: $y = 0.667 + 1.5x$

Observed data points

9 Statistics Formulas (Alternative Calculation)

$$m = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

$$b = \bar{y} - m\bar{x}$$

Mean of x: $\bar{x} = \frac{1+2+3}{3} = 2$

Mean of y: $\bar{y} = \frac{2+4+5}{3} = \frac{11}{3} \approx 3.667$

Using these formulas:

- $m = 1.5$
- $b \approx 0.667$

➡ Same result as OLS!

Machine Learning Notes

Page 3 — K Nearest Neighbors (KNN)

1) Definition

K Nearest Neighbors (KNN) is a distance-based classification algorithm.
It is a lazy learner (no explicit training model).

★ An odd value of k is often chosen to reduce the chance of ties between classes.

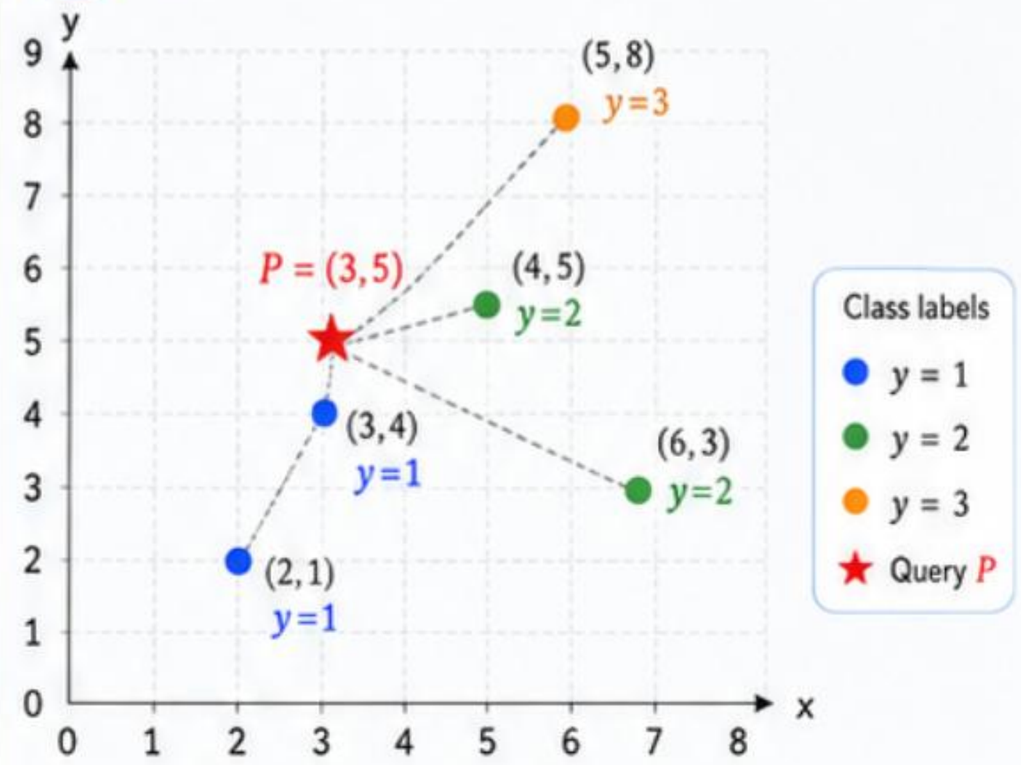
2) Example Data

```
X = np.array([[2,1], [3,4], [6,3],
              [4,5], [5,8]])
y = np.array([1, 1, 2, 2, 3])
```

Query point: $P = (3, 5)$

Index	1	2	3	4	5
$X_i\ (x, y)$	(2,1)	(3,4)	(6,3)	(4,5)	(5,8)
y_i (label)	1	1	2	2	3

3) Scatter Plot Illustration



KNN looks at the k closest training points to P and predicts the most common class.

Euclidean Distance

Formula

$$d = \sqrt{(x_2 - x_1)^2 + (y_2 - y_1)^2}$$

Distances from P = (3, 5)

to (2, 1)	$= \sqrt{(2-3)^2 + (1-5)^2} = \sqrt{17} \approx 4.12$
to (3, 4)	$= \sqrt{(3-3)^2 + (4-5)^2} = \sqrt{1} = 1.00$
to (6, 3)	$= \sqrt{(6-3)^2 + (3-5)^2} = \sqrt{13} \approx 3.61$
to (4, 5)	$= \sqrt{(4-3)^2 + (5-5)^2} = \sqrt{1} = 1.00$
to (5, 8)	$= \sqrt{(5-3)^2 + (8-5)^2} = \sqrt{13} \approx 3.61$

Nearest Neighbor Intuition

Smaller distance
⇒ closer neighbor.
The k closest points are used to determine the class of P.

Example (k = 3)
Closest 3 neighbors to P:
(3,4) [1], (4,5) [2], (6,3) [2]
Predicted class for P = 2

Distance Metrics

Manhattan Distance

$$d = |x_2 - x_1| + |y_2 - y_1|$$

(L_1 norm)

Example (P = (3,5))
to (2,1): |2−3| + |1−5| = 1 + 4 = 5
to (4,5): |4−3| + |5−5| = 1 + 0 = 1

Hamming Distance

Number of positions that are different.
Used for binary / categorical data.

Example
(0,1,0,1) vs (1,1,0,0)
Hamming distance = 2

Minkowski Distance

$$d = \left(|x_2 - x_1|^p + |y_2 - y_1|^p \right)^{\frac{1}{p}}$$

$(p \geq 1)$

General form; includes Manhattan and Euclidean.

Special Cases

If $p = 1$
⇒ Minkowski becomes Manhattan distance.

If $p = 2$
⇒ Minkowski becomes Euclidean distance.

Summary

KNN Workflow

1. Store training data (X, y) →
2. Compute distances to query point P →
3. Select k nearest neighbors →
4. Majority vote (classification)

7) At a Glance: Linear Regression vs KNN

Linear Regression	→	Regression (continuous target)
K Nearest Neighbors (KNN)	→	Distance-based classification